

Q1)

1)

	0	1	0	1	1	0	1	1	0
1	0	1	1	1	1	1	1	1	1
0	1	1	2	2	2	2	2	2	2
0	1	1	2	2	2	3	3	3	3
1	1	2	2	3	3	3	4	4	4
0	1	2	3	3	3	4	4	4	5
1	1	2	3	4	4	4	5	5	5
0	1	2	3	4	4	5	5	5	6
1	1	2	3	4	5	5	6	6	6

2) The Length of the LCS is $dp[8][9] = 6$

3) Starting from the bottom-right cell $L[8][9]$:

(8,9): $X_8 = 1, Y_9 = 0 \rightarrow$ no match, move up to (7,9)

(7,9): $X_7 = 0, Y_9 = 0 \rightarrow$ match, add 0, move diagonal to (6,8)

(6,8): $X_6 = 1, Y_8 = 1 \rightarrow$ match, add 1, move diagonal to (5,7)

(5,7): $X_5 = 0, Y_7 = 1 \rightarrow$ no match, move up to (4,7)

(4,7): $X_4 = 1, Y_7 = 1 \rightarrow$ match, add 1, move diagonal to (3,6)

(3,6): $X_3 = 0, Y_6 = 0 \rightarrow$ match, add 0, move diagonal to (2,5)

(2,5): $X_2 = 0, Y_5 = 1 \rightarrow$ no match, move left to (2,4)

(2,4): $X_2 = 0, Y_4 = 1 \rightarrow$ no match, move left to (2,3)

(2,3): $X_2 = 0, Y_3 = 0 \rightarrow$ match, add 0, move diagonal to (1,2)

(1,2): $X_1 = 1, Y_2 = 1 \rightarrow$ match, add 1, move diagonal to (0,1)

Stop.

Collected sequence (reverse order): 011001

Final LCS after reversing: 100110

One possible LCS is: 100110

Q2)

- 1) The main concept is to gradually construct the solution with the help of dynamic programming. We find out how many different paths lead to every score starting from 0 until n by using the provided move values. Rather than re-evaluating the subproblems every time, we keep the count of ways for each intermediate score and leverage those outcomes.
- 2) DP Table Explanation (using 1D DP)

We use a 1D DP array of size $n+1$.

$dp[s]$ represents the number of ways to reach score s using the available move values.

- The array runs from 0 to n .
- We initialize $dp[0] = 1$, because there is only one way to make a score of 0 by taking no moves.
- For each move p , we update the dp array so that $dp[i]$ adds the number of ways to reach score i , including move p .

Each subproblem $dp[i]$ therefore represents:

“How many ways can we form score i using all move values considered so far?”

- 3) For each move value p , for each score $s \geq p$: $dp[s] = dp[s] + dp[s-p]$
- 4) Explanation of Recurrence

If we want to compute $dp[s]$ (ways to reach score s):

- $dp[s]$ already contains all ways that do not use the move value p
- $dp[s-p]$ represents all ways to reach the remaining score after using one p .

So we add them:

- the ways without using p
- Plus the ways where we do use p at least once

This counts all valid combinations that sum to s .

5)

```
Algorithm CountWays(points[], n):
    create array dp[0..n]

    for s from 0 to n:
        dp[s] = 0

    dp[0] = 1    # one way to make score 0

    for each p in points:
        for s from p to n:
            dp[s] = dp[s] + dp[s - p]

    return dp[n]
```

- 6) We loop over each move value, and for each one, we loop through scores up to n .

Thus, the running time is:

$O(k*n)$ where

- k = number of move values
- n = target score

This is efficient and optimal for this problem

Q3)

- 1) Dynamic programming is applied to get the optimal total value for every possible box capacity, starting from 0 until W . For each capacity, the addition of each pastry type (if it fits) is tried, and the maximum value produced by that choice is determined. Since it is allowed to use pastry types repeatedly, this situation is viewed as an unbounded knapsack problem: the optimal solution for capacity c can be derived from the optimal solutions of lesser capacities $c - w_i$.
- 2) We use a 1D DP array of size $W+1$,
 $dp[c]$ = the maximum total value of pastries that can be packed in the box with capacity strictly c (or at most c , given that the addition of extra weight is valuable only in terms of the increase in value).
Base case: $dp[0] = 0$, since with 0 capacity we can't take any pastries, so the total value is 0.
Each subproblem $dp[c]$ answers, What is the maximum value we can achieve with a box capacity of c pounds using any number of copies of each pastry type?
- 3) $dp[c] = \sum dp[c-p]$ if $c \geq p$
each $p \in \text{points}$
- 4) To compute the best value for capacity c , we consider each pastry type i that fits; if we do not take pastry i , the best we can do is already stored in $dp[c]$.
 - In case we take one unit of pastry i , our capacity will be reduced by w_i , and we will get v_i as a profit, and in addition to that, we can have the best value with the remaining capacity $c - w_i$, and that will be $dp[c - w_i]$.

Thus, the ideal selection for capacity c is the optimum among all such options: either retaining the present best ($dp[c]$) or incorporating pastry i to the best resolution of $c - w_i$. Since it is possible to use each pastry category any number of times, we can take $dp[c - w_i]$ with the same item again.

5)

```
Algorithm MaxPastryValue(weights[1..n], values[1..n], W):  
    # weights[i] = w_i, values[i] = v_i, box capacity = W  
    create array dp[0..W]  
  
    # base case: 0 capacity -> 0 value
```

```

for c from 0 to W:
    dp[c] = 0

# unbounded knapsack DP
for c from 1 to W:                # for each capacity
    for i from 1 to n:            # try each pastry type
        if weights[i] <= c:
            candidate = dp[c - weights[i]] + values[i]
            if candidate > dp[c]:
                dp[c] = candidate

return dp[W]

```

6) We have two nested loops:

- Outer loop over capacities c from 1 to $W \rightarrow W$ iterations.
- Inner loop over pastry types i from 1 to $n \rightarrow n$ iterations.

Each iteration is $O(1)$, so the total running time is:

$O(n*W)$. The space usage is $O(W)$ for the 1D dp array.